# INTRODUCTION TO C++

## WORKSHEET 2



*Submitted By:* **Sadhabi Dev**
***Level*** : 4
***Student ID*** : 23085140
Cyber Security and Digital Forensics

## QUESTION NO.1

Write a program that manages a simple student grade calculator with the following requirements. Create a Student class that has:

- Student name (string)
- Three subject marks (integers)
- A basic member function to calculate average

The program should:

1. Accept student details (name and marks) from user input
2. Calculate and display:
- Total marks
- Average marks
- Grade (A for ≥90%, B for ≥80%, C for ≥70%, D for ≥60%, F for <60%)
- Display a message if any mark is below 0 or above 100

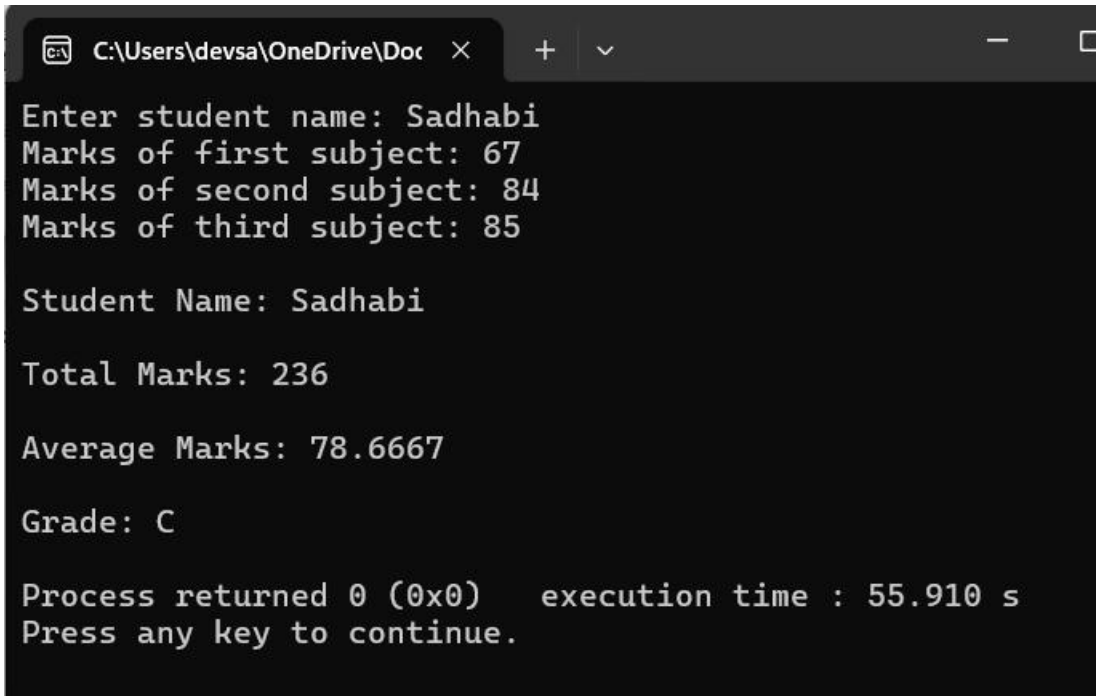
## SOLUTION:

```
#include <iostream>
using namespace std;

class Student {
public:
   string studentName;
   int m1, m2, m3;
   int totalMarks;
   char grade;
   float average;

   void getInfo() {
      cout << "Enter student name: ";
      cin >> studentName;

      cout << "Marks of first subject: ";
      cin >> m1;

      cout << "Marks of second subject: ";
      cin >> m2;

      cout << "Marks of third subject: ";
      cin >> m3;


      if (m1 < 0 || m1 > 100 || m2 < 0 || m2 > 100 || m3 < 0 || m3 > 100) {
            cout << " ";
         cout << "\nInvalid Input! Marks must be between 0 and 100" << endl;
      }
   }

   void calculateTotal() {
```

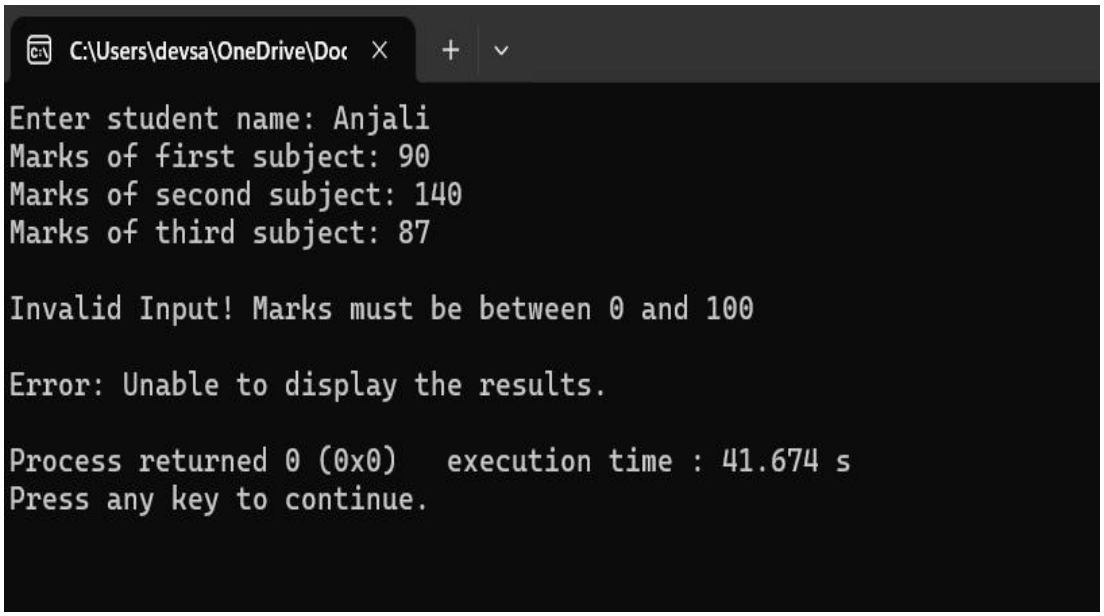
```
        totalMarks = m1 + m2 + m3;
    }

    void calculateGrade() {
        average = totalMarks / 3.0;

        if (average >= 90) {
            grade = 'A';
        } else if (average >= 80) {
            grade = 'B';
        } else if (average >= 70) {
            grade = 'C';
        } else if (average >= 60) {
            grade = 'D';
        } else {
            grade = 'F';
        }
    }

    void displayResults() {
        if (m1 >= 0 && m1 <= 100 && m2 >= 0 && m2 <= 100 && m3 >= 0 && m3
<= 100) {
            calculateTotal();
            calculateGrade();

            cout << "\nStudent Name: " << studentName << endl;
            cout << "\nTotal Marks: " << totalMarks << endl;
            cout << "\nAverage Marks: " << average << endl;
            cout << "\nGrade: " << grade << endl;
        } else {
            cout << "\nError: Unable to display the results." << endl;
        }
    }
};

int main() {
    Student student;
    student.getInfo();
    student.displayResults();

    return 0;
}
```

**Output of Basic Student Grading System.**

**a. While giving the valid input to user**




**b. While giving invalid input to the user.**




## QUESTION NO.2

Write a program with a class Circle having:

- Private member: radius (float)
- A constructor to initialize radius
- A friend function compareTwoCircles that takes two Circle objects and prints which circle has the larger area

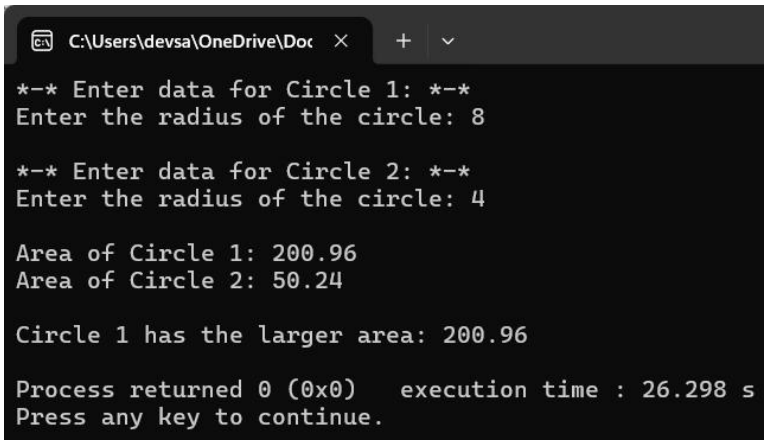
## SOLUTION:

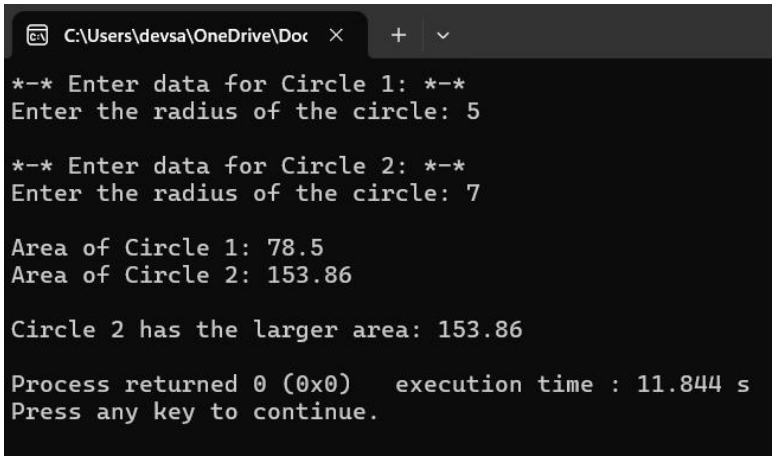
```
#include <iostream>
using namespace std;

class Circle {

private:
   float radius;

public:
   Circle(float r = 0) {
      radius = r;
   }

   float getArea() {
      return 3.14 * radius * radius;
   }

   void getData() {
      cout << "Enter the radius of the circle: ";
      cin >> radius;
   }

   friend void compareTwoCircles(Circle c1, Circle c2);
};

void compareTwoCircles(Circle c1, Circle c2) {

   float a1 = c1.getArea();
   float a2 = c2.getArea();

   cout << "\nArea of Circle 1: " << a1 << endl;
   cout << "Area of Circle 2: " << a2 << endl;

   if (a1 > a2) {
      cout << "\nCircle 1 has the larger area: " << a1 << endl;
   } else if (a2 > a1) {
      cout << "\nCircle 2 has the larger area: " << a2 << endl;
   } else {
      cout << "\nBoth circles have the same area." << endl;
   }
}

int main() {
   Circle c1, c2;

   cout << "*-* Enter data for Circle 1: *-*" << endl;
   c1.getData();
```

```
    cout << "\n*-* Enter data for Circle 2: *-*" << endl;
    c2.getData();

    compareTwoCircles(c1, c2);

    return 0;
}
```

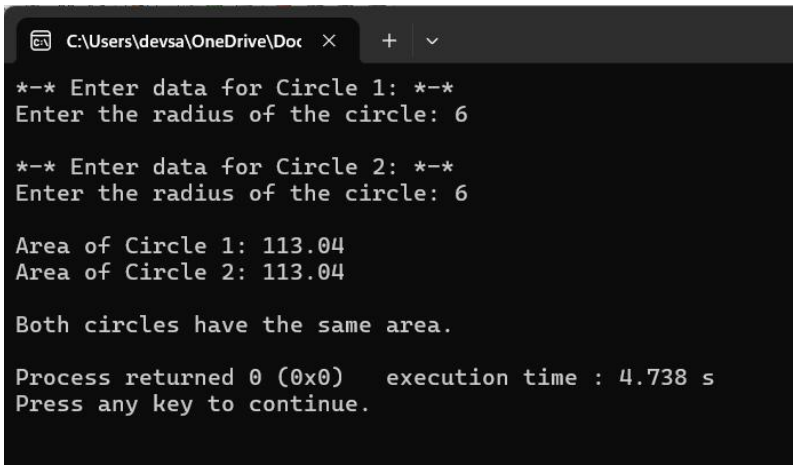
**The output of Circle class and Overloaded functions:**

**a. When the circles have different radius:**




**b. When circles have same radius.**



## QUESTION NO.3

Create a program with these overloaded functions named findMax:

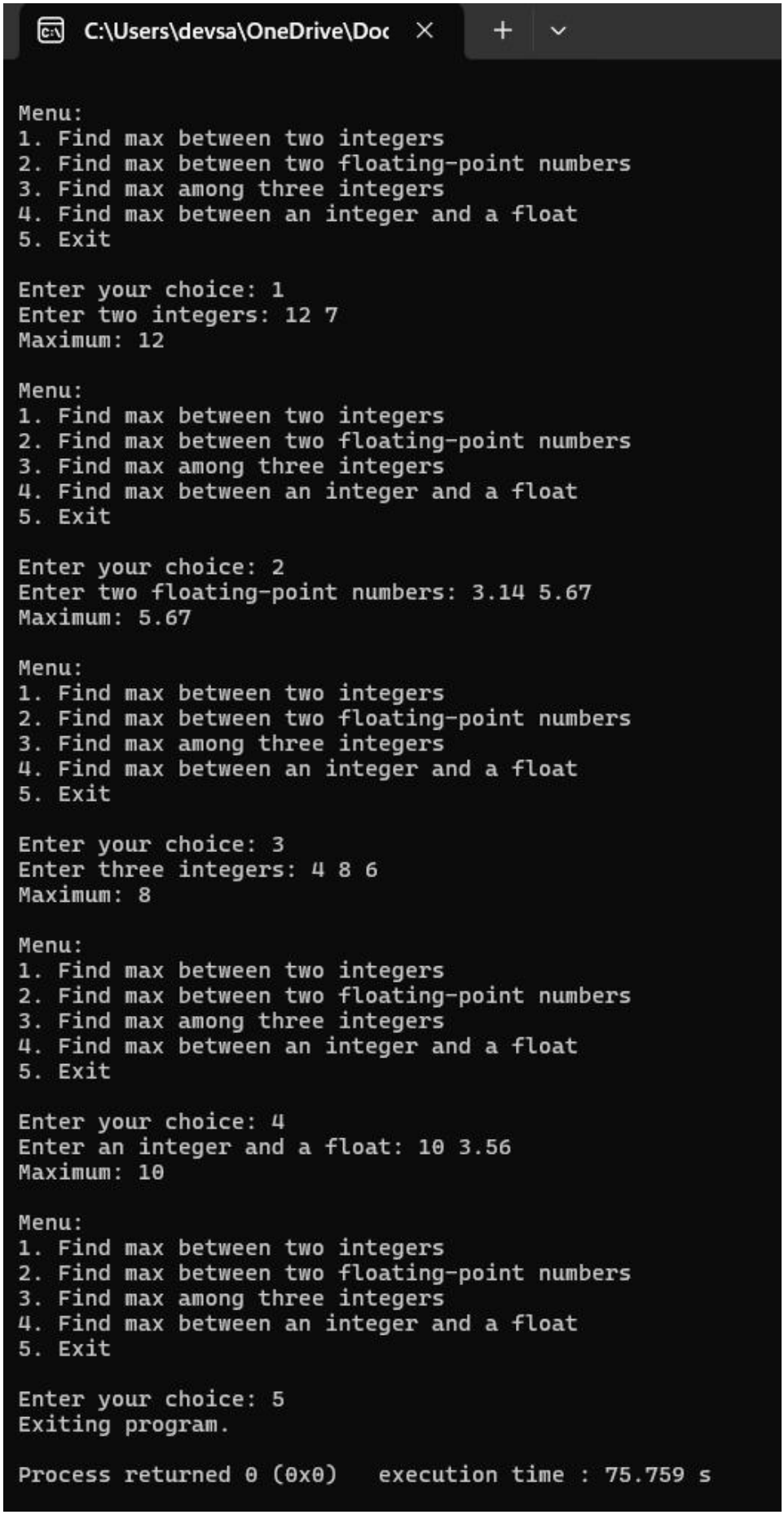
- One that finds maximum between two integers
- One that finds maximum between two floating-point numbers
- One that finds maximum among three integers
- One that finds maximum between an integer and a float

## SOLUTION:

```
#include <iostream>
using namespace std;

class MaxFinder {
public:
   int findMax(int a, int b) {
      return (a > b) ? a : b;
   }

   float findMax(float a, float b) {
      return (a > b) ? a : b;
   }

   int findMax(int a, int b, int c) {
      return (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
   }

   float findMax(int a, float b) {
      return (a > b) ? a : b;
   }
};

void menu() {
   MaxFinder m1;
   int choice;

   do {
      cout << "\nMenu:";
      cout << "\n1. Find max between two integers";
      cout << "\n2. Find max between two floating-point numbers";
      cout << "\n3. Find max among three integers";
      cout << "\n4. Find max between an integer and a float";
      cout << "\n5. Exit";
      cout << "\n\nEnter your choice: ";
      cin >> choice;

      switch (choice) {
         case 1: {
            int a, b;
```

```
            cout << "Enter two integers: ";
            cin >> a >> b;
            cout << "Maximum: " << m1.findMax(a, b) << endl;
            break;
        }
        case 2: {
            float a, b;
            cout << "Enter two floating-point numbers: ";
            cin >> a >> b;
            cout << "Maximum: " << m1.findMax(a, b) << endl;
            break;
        }
        case 3: {
            int a, b, c;
            cout << "Enter three integers: ";
            cin >> a >> b >> c;
            cout << "Maximum: " << m1.findMax(a, b, c) << endl;
            break;
        }
        case 4: {
            int a;
            float b;
            cout << "Enter an integer and a float: ";
            cin >> a >> b;
            cout << "Maximum: " << m1.findMax(a, b) << endl;
            break;
        }
        case 5:
            cout << "Exiting program.\n";
            break;
        default:
            cout << "Invalid choice! Try again.\n";
      }
   } while (choice != 5);
}

int main() {
   menu();
   return 0;
}
```

**The output of finding the maximum number between integers and floating point numbers.**


C:\Users\devsa\OneDrive\Doc
Menu:
1. Find max between two integers
2. Find max between two floating-point numbers
3. Find max among three integers
4. Find max between an integer and a float
5. Exit
Enter your choice: 1
Enter two integers: 12 7
Maximum: 12
Menu:
1. Find max between two integers
2. Find max between two floating-point numbers
3. Find max among three integers
4. Find max between an integer and a float
5. Exit
Enter your choice: 2
Enter two floating-point numbers: 3.14 5.67
Maximum: 5.67
Menu:
1. Find max between two integers
2. Find max between two floating-point numbers
3. Find max among three integers
4. Find max between an integer and a float
5. Exit
Enter your choice: 3
Enter three integers: 4 8 6
Maximum: 8
Menu:
1. Find max between two integers
2. Find max between two floating-point numbers
3. Find max among three integers
4. Find max between an integer and a float
5. Exit
Enter your choice: 4
Enter an integer and a float: 10 3.56
Maximum: 10
Menu:
1. Find max between two integers
2. Find max between two floating-point numbers
3. Find max among three integers
4. Find max between an integer and a float
5. Exit
Enter your choice: 5
Exiting program.
Process returned 0 (0x0) execution time : 75.759 s

## QUESTION NO.4

Write a program that reads the titles of 10 books (use an array of 150 characters) and writes them in a binary file selected by the user.
The program should read a title and display a message to indicate if it is contained in the file or not.

Create a program that:
- Reads student records (roll, name, marks) from a text file
- Throws an exception if marks are not between 0 and 100
- Allows adding new records with proper validation
- Saves modified records back to file

## SOLUTION:

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

using namespace std;

class Student {
public:
    int roll;
    string name;
    int marks;

    bool validateMarks() {
        return marks >= 0 && marks <= 100;
    }
};

class StudentManager {
public:
    void read(string filename, vector<Student>& students) {
        ifstream infile(filename);
        if (infile.fail()) {
            cout << "\nError opening file for reading\n";
            return;
        }

        students.clear();

        Student s;
        while (infile >> s.roll) {
            infile.ignore();
            getline(infile, s.name, ' ');
            infile >> s.marks;
```

```
            students.push_back(s);
        }
        infile.close();
    }

    void write(vector<Student>& students) {
        Student s;
        cout << "\nEnter student roll number: ";
        cin >> s.roll;
        cout << "Enter student name: ";
        cin.ignore();
        getline(cin, s.name);
        cout << "Enter student marks: ";
        cin >> s.marks;

        if (!s.validateMarks()) {
            cout << "Marks must be between 0 and 100\n";
            return;
        }
        students.push_back(s);
        cout << "Student record added successfully!\n";
    }

    void save(string filename, vector<Student>& students) {
        ofstream outfile(filename);
        if (outfile.fail()) {
            cout << "Error opening file for writing\n";
            return;
        }

        for (auto& s : students) {
            outfile << s.roll << " " << s.name << " " << s.marks << "\n";
        }
        outfile.close();
        cout << "Records saved to file successfully!\n";
    }

    void displayRecords(vector<Student>& students) {
        if (students.empty()) {
            cout << "\nNo student records available.\n";
            return;
        }

        cout << "\n|================================================|";
        cout << "\n|            Student Records                     |";
        cout << "\n|================================================|";
        for (auto& s : students) {
            cout << "\nRoll No: " << s.roll << " | Name: " << s.name << " | Marks: " <<
s.marks;
        }
```

```
        cout <<
"\n|================================================|\n";
    }
};

class BookManager {
public:
    void writeBooks(string filename, string titles[], int count) {
        ofstream outfile(filename, ios::binary);
        if (outfile.fail()) {
            cout << "Error opening file for writing\n";
            return;
        }

        for (int i = 0; i < count; ++i) {
            outfile.write(titles[i].c_str(), titles[i].length() + 1);
        }
        outfile.close();
    }

    bool readBook(string filename, string title) {
        ifstream infile(filename, ios::binary);
        if (infile.fail()) {
            cout << "Error opening file for reading\n";
            return false;
        }

        string storedtitle;
        while (getline(infile, storedtitle, '\0')) {
            if (storedtitle == title) {
                return true;
            }
        }
        return false;
    }
};

void showMenu() {
    cout << "\n\n************************************************";
    cout << "\n               Main Menu";
    cout << "\n************************************************";
    cout << "\n1) Add New Student Record";
    cout << "\n2) Save and Exit";
    cout << "\n3) Show Student Records";
    cout << "\n4) Add New Book Titles";
    cout << "\n5) Search for a Book";
    cout << "\n************************************************";
    cout << "\nPlease enter your choice: ";
}
```

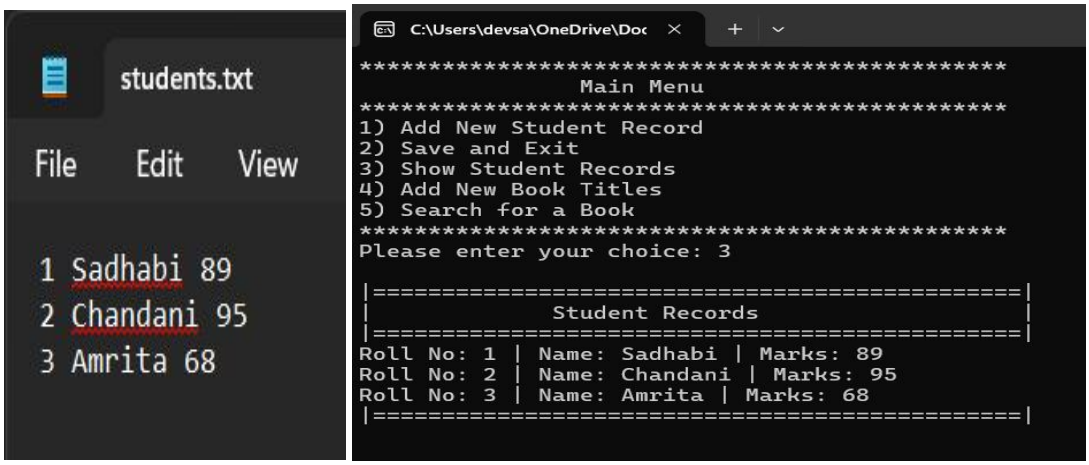
```
int main() {
    vector<Student> students;
    string studentfile = "students.txt";

    StudentManager studentManager;
    studentManager.read(studentfile, students);

    int choice;
    BookManager bookManager;
    string bookfile = "books.dat";
    int bookcount = 10;
    string books[bookcount];

    do {
        showMenu();
        cin >> choice;
        cin.ignore();

        switch (choice) {
            case 1:
                studentManager.write(students);
                break;
            case 2:
                studentManager.save(studentfile, students);
                cout << "Records saved to file!\n";
                break;
            case 3:
                studentManager.displayRecords(students);
                break;
            case 4:
                cout <<
"\n|================================================|";
                cout << "\n|           Enter 10 Book Titles             |";
                cout <<
"\n|================================================|";
                for (int i = 0; i < bookcount; ++i) {
                    cout << "\nBook " << i + 1 << ": ";
                    getline(cin, books[i]);
                }
                bookManager.writeBooks(bookfile, books, bookcount);
                cout << "Books written to binary file successfully.\n";
                break;
            case 5: {
                string searchtitle;
                char searchagain = 'y';
                while (searchagain == 'y' ) {
                    cout << "\nEnter a book title to search: ";
                    getline(cin, searchtitle);

                    if (bookManager.readBook(bookfile, searchtitle)) {
```

```
                cout << "The book is in the file.\n";
            } else {
                cout << "The book is not in the file.\n";
            }

            cout << "Do you want to search for another book? (y/n): ";
            cin >> searchagain;
            cin.ignore();
        }
        break;
    }
    default:
        cout << "Invalid choice! Please try again.\n";
  }
} while (choice != 2);

cout << "\nThank you for using the system!\n";
return 0;
}
```

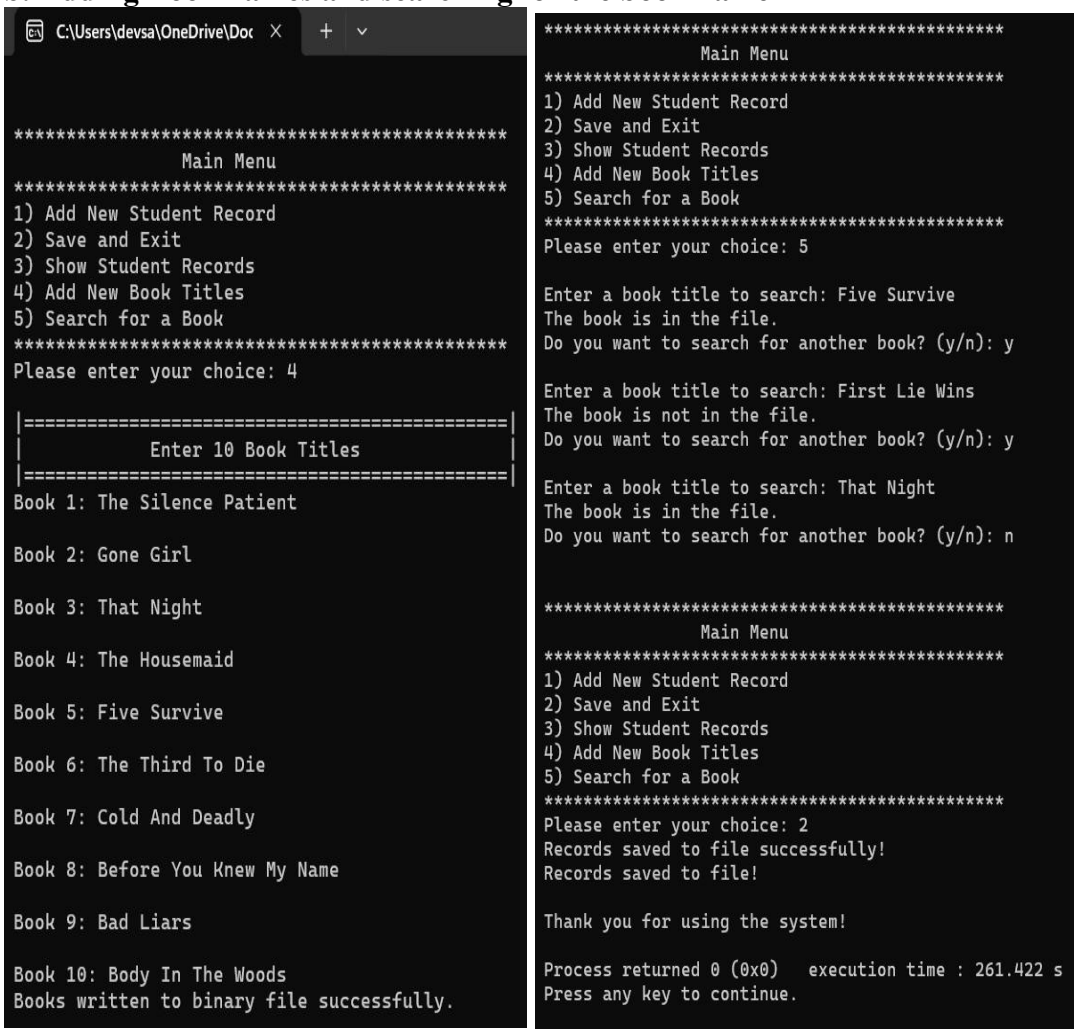
**The output of the days of the week based on user input .**

**a. Adding Students Record and saving it.**

**The student record added has been saved in the student.txt file and rechecking if the record have been added to the system or not as shown below.**




**b. Adding Book names and searching for the book name**




**My Github Link: https://github.com/sd6012/cpp_Assignment**